Lab 03 · Pipeline completa con CodePipeline

Duración estimada: 60 minutos **Sesión:** 2 **Entorno:** GitHub Codespaces con un emulador local de AWS · sin cuenta de AWS

Objetivo

Construir una pipeline de entrega continua con cinco etapas: **Source** (zip en S3) → **Build** (estilo y pruebas con CodeBuild) → **Deploy-Pre** → **Aprobación** manual → **Deploy-Pro**. Después publicarás versiones nuevas y las seguirás de principio a fin: una que llega a producción, otra que rechazas en la aprobación y otra que la propia pipeline detiene porque rompe las pruebas.

Cómo funciona el entorno

Trabajas en el mismo Codespace que en los labs anteriores. El emulador ejecuta la pipeline de verdad: vigila el zip del bucket de origen, lanza el build en un contenedor, copia el artefacto a los buckets de preproducción y producción y se detiene en la etapa de aprobación hasta que alguien la aprueba o la rechaza.

> **Diferencias con AWS real en esta práctica.** En AWS real, la acción de Build recibe el artefacto de la etapa Source y la etapa de despliegue suele ser CloudFormation (como en el Lab 01). El emulador no entrega el artefacto de la pipeline a CodeBuild ni soporta la acción de despliegue de CloudFormation, así que hacemos dos ajustes: el proyecto de CodeBuild lee el código **del mismo zip del bucket de origen**, y los despliegues copian el artefacto a dos buckets S3 (`pre` y `pro`), que es una acción de despliegue real de CodePipeline (la usan, por ejemplo, las webs estáticas). La lógica de la pipeline —qué etapa bloquea a cuál, la aprobación, el sondeo del origen— es la misma. Al final tienes cómo sería la pipeline en AWS real.

Requisitos

- El Codespace del curso abierto y el emulador respondiendo: `aws sts get-caller-identity`.
- Haber hecho el Lab 02 ayuda, porque el buildspec es del mismo estilo.

Punto de partida

La práctica es independiente. En el paso 1 despliegas con CloudFormation los cuatro buckets y los roles de servicio.

Paso 1 · Desplegar el punto de partida

```
mkdir -p ~/lab03 && cd ~/lab03

cat > inicio-lab03.yaml <<'EOF2'
AWSTemplateFormatVersion: "2010-09-09"
Description: Lab 03 - punto de partida de la pipeline
Resources:
  BucketFuente:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: { "Fn::Sub": "lab03-fuente-${AWS::AccountId}" }
      VersioningConfiguration: { Status: Enabled }   # obligatorio para el origen S3
  BucketArtefactos:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: { "Fn::Sub": "lab03-artefactos-${AWS::AccountId}" }
  BucketPre:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: { "Fn::Sub": "lab03-pre-${AWS::AccountId}" }
  BucketPro:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: { "Fn::Sub": "lab03-pro-${AWS::AccountId}" }
  RolCodeBuild:
```

```
      Type: AWS::IAM::Role
      Properties:
        RoleName: lab03-codebuild
        AssumeRolePolicyDocument:
          Version: "2012-10-17"
          Statement:
            - { Effect: Allow, Principal: { Service: codebuild.amazonaws.com }, Action: sts:AssumeRole }
        Policies:
          - PolicyName: permisos-build
            PolicyDocument:
              Version: "2012-10-17"
              Statement:
                - Effect: Allow
                  Action: [logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents]
                  Resource: "*"
                - Effect: Allow
                  Action: [s3:GetObject, s3:GetObjectVersion, s3:PutObject, s3:ListBucket]
                  Resource:
                    - { "Fn::GetAtt": [BucketFuente, Arn] }
                    - { "Fn::Sub": "${BucketFuente.Arn}/*" }
                    - { "Fn::GetAtt": [BucketArtefactos, Arn] }
                    - { "Fn::Sub": "${BucketArtefactos.Arn}/*" }
  RolPipeline:
    Type: AWS::IAM::Role
    Properties:
      RoleName: lab03-pipeline
      AssumeRolePolicyDocument:
        Version: "2012-10-17"
        Statement:
          - { Effect: Allow, Principal: { Service: codepipeline.amazonaws.com }, Action: sts:AssumeRole }
      Policies:
        - PolicyName: permisos-pipeline
          PolicyDocument:
            Version: "2012-10-17"
            Statement:
              - Effect: Allow
                Action: [s3:GetObject, s3:GetObjectVersion, s3:GetBucketVersioning, s3:PutObject, s3:ListBuc
                Resource: "*"
              - Effect: Allow
                Action: [codebuild:StartBuild, codebuild:BatchGetBuilds]
                Resource: { "Fn::Sub": "arn:aws:codebuild:${AWS::Region}:${AWS::AccountId}:project/lab03-*"
Outputs:
  Fuente: { Value: { Ref: BucketFuente } }
  Artefactos: { Value: { Ref: BucketArtefactos } }
  Pre: { Value: { Ref: BucketPre } }
  Pro: { Value: { Ref: BucketPro } }
  RolCodeBuild: { Value: { "Fn::GetAtt": [RolCodeBuild, Arn] } }
  RolPipeline: { Value: { "Fn::GetAtt": [RolPipeline, Arn] } }
EOF2

cfn-lint inicio-lab03.yaml && echo "Plantilla correcta"

aws cloudformation deploy --stack-name lab03-inicio \
  --template-file inicio-lab03.yaml --capabilities CAPABILITY_NAMED_IAM

salida() { aws cloudformation describe-stacks --stack-name lab03-inicio \
  --query "Stacks[0].Outputs[?OutputKey=='$1'].OutputValue" --output text; }
export FUENTE=$(salida Fuente) ARTEFACTOS=$(salida Artefactos) PRE=$(salida Pre) PRO=$(salida Pro)
export ROL_BUILD=$(salida RolCodeBuild) ROL_PIPE=$(salida RolPipeline)
env | grep -E "^(FUENTE|ARTEFACTOS|PRE|PRO|ROL_)" | sort
```

> **Punto de control 1.** Ves seis variables con valores. Si abres una terminal nueva, vuelve a ejecutar la función `salida` y los `export` de este bloque.

Paso 2 · Aplicación, pruebas y buildspec

La aplicación es un pequeño servicio de saludo con su versión. El build hace lo mismo que en el Lab 02: estilo con `ruff` y pruebas con `pytest`.

```
        mkdir -p app/src app/tests && cd app

        cat > src/app.py <<'EOF2'
        VERSION = "1.0.0"


        def saludo(nombre: str) -> str:
            return f"Hola, {nombre}"


        def info() -> dict:
            return {"servicio": "saludos", "version": VERSION}
        EOF2
        touch src/__init__.py

        cat > tests/test_app.py <<'EOF2'
        from src.app import info, saludo


        def test_saludo():
            assert saludo("Ana") == "Hola, Ana"


        def test_info_tiene_version():
            assert info()["version"].count(".") == 2
        EOF2

        cat > requirements-dev.txt <<'EOF2'
        pytest==8.3.3
        ruff==0.6.9
        EOF2

        cat > buildspec.yml <<'EOF2'
        version: 0.2
        env:
          shell: bash
        phases:
          install:
            commands:
              - set -o pipefail; pip install --quiet --root-user-action=ignore --disable-pip-version-check -r
requirements-dev.txt 2>&1 | tee -a build.log
          pre_build:
            commands:
              - set -o pipefail; ruff check src tests 2>&1 | tee -a build.log
          build:
            commands:
              - set -o pipefail; python -m pytest -v 2>&1 | tee -a build.log
        artifacts:
          files:
            - build.log
        EOF2

        pip install --quiet -r requirements-dev.txt && python -m pytest -q && ruff check src tests
```

Define dos funciones que usarás durante toda la práctica: `publicar` empaqueta el código y lo sube al bucket de origen (eso es lo que dispara la pipeline) y `version_en` te dice qué versión hay desplegada en un bucket.

```
        publicar() {
          (cd ~/lab03/app && rm -f ../app.zip && python -m zipfile -c ../app.zip src tests requirements-dev.txt
buildspec.yml)
          aws s3 cp ~/lab03/app.zip s3://$FUENTE/fuente/app.zip
        }

        version_en() {
          CLAVE=$(aws s3 ls s3://$1 --recursive | grep "\.zip$" | sort | tail -1 | awk '{print $4}')
          if [ -z "$CLAVE" ]; then echo "$1: (vacío)"; return; fi
          aws s3 cp s3://$1/$CLAVE /tmp/desplegado.zip --quiet
          echo "$1/$CLAVE -> $(python -c "import zipfile;
print(zipfile.ZipFile('/tmp/desplegado.zip').read('src/app.py').decode().splitlines()[0])")"
        }
```

```
publicar
cd ~/lab03
```

Paso 3 · Crear el proyecto de CodeBuild

```
aws codebuild create-project --name lab03-pruebas \
  --source type=S3,location=$FUENTE/fuente/app.zip \
  --artifacts type=S3,location=$ARTEFACTOS,name=build,packaging=NONE \
  --environment type=LINUX_CONTAINER,image=python:3.12-slim,computeType=BUILD_GENERAL1_SMALL \
  --service-role $ROL_BUILD \
  --query "project.[name,source.location]" --output text
```

Paso 4 · Crear la pipeline

Una pipeline se define en un documento JSON: un rol, un almacén de artefactos y una lista de etapas. Cada etapa tiene acciones, y cada acción declara qué artefactos consume (`inputArtifacts`) y cuáles produce (`outputArtifacts`). Así es como la pipeline sabe que **el mismo zip** que entra por Source es el que llega a preproducción y, después, a producción.

```
cat > pipeline.json <<EOF2
{
  "pipeline": {
    "name": "lab03",
    "roleArn": "$ROL_PIPE",
    "pipelineType": "V2",
    "executionMode": "QUEUED",
    "artifactStore": { "type": "S3", "location": "$ARTEFACTOS" },
    "stages": [
      { "name": "Source", "actions": [ {
          "name": "Fuente",
          "actionTypeId": { "category": "Source", "owner": "AWS", "provider": "S3", "version": "1" },
          "configuration": { "S3Bucket": "$FUENTE", "S3ObjectKey": "fuente/app.zip", "PollForSourceChanges":
"true" },
          "outputArtifacts": [ { "name": "Codigo" } ] } ] },
      { "name": "Build", "actions": [ {
          "name": "Pruebas",
          "actionTypeId": { "category": "Build", "owner": "AWS", "provider": "CodeBuild", "version": "1" },
          "configuration": { "ProjectName": "lab03-pruebas" },
          "inputArtifacts": [ { "name": "Codigo" } ] } ] },
      { "name": "Deploy-Pre", "actions": [ {
          "name": "PublicarPre",
          "actionTypeId": { "category": "Deploy", "owner": "AWS", "provider": "S3", "version": "1" },
          "configuration": { "BucketName": "$PRE", "Extract": "false", "ObjectKey": "app.zip" },
          "inputArtifacts": [ { "name": "Codigo" } ] } ] },
      { "name": "Aprobacion", "actions": [ {
          "name": "Aprobar",
          "actionTypeId": { "category": "Approval", "owner": "AWS", "provider": "Manual", "version": "1" },
          "configuration": { "CustomData": "Revisa la versión desplegada en preproducción antes de aprobar"
},
      { "name": "Deploy-Pro", "actions": [ {
          "name": "PublicarPro",
          "actionTypeId": { "category": "Deploy", "owner": "AWS", "provider": "S3", "version": "1" },
          "configuration": { "BucketName": "$PRO", "Extract": "false", "ObjectKey": "app.zip" },
          "inputArtifacts": [ { "name": "Codigo" } ] } ] }
    ]
  }
}
EOF2

aws codepipeline create-pipeline --cli-input-json file://pipeline.json --query "pipeline.[name,pipelineType]
output text
```

Elemento	Valor	Para qué sirve
`executionMode`	`QUEUED`	Si llega un cambio mientras otro está en marcha, espera su turno en lugar de pisarlo.
`artifactStore`	bucket de artefactos	Donde la pipeline guarda lo que pasa de una etapa a otra.

`PollForSourceChanges`	`true`	La pipeline vigila el zip y arranca sola cuando cambia. En AWS real se prefiere un evento de EventBridge.
`Approval / Manual`	etapa `Aprobacion`	Detiene la ejecución hasta que una persona decide.

Para seguir la pipeline usarás tres funciones: `estado` enseña cómo está cada etapa, `esperar` aguarda a que una etapa llegue a un estado concreto y `esperar_version` aguarda a que una versión llegue a un entorno.

```
estado() {
  aws codepipeline get-pipeline-state --name lab03 \
    --query "stageStates[].[stageName,latestExecution.status]" --output table
}

esperar() {   # uso: esperar <etapa> <estado>
  echo -n "Esperando a que $1 esté en $2 "
  for i in $(seq 1 100); do
    S=$(aws codepipeline get-pipeline-state --name lab03 \
      --query "stageStates[?stageName=='$1'].latestExecution.status" --output text)
    [ "$S" = "$2" ] && break
    echo -n "."; sleep 3
  done
  echo; estado
}

esperar_version() {   # uso: esperar_version <bucket> <versión>
  echo -n "Esperando a que llegue la $2 "
  for i in $(seq 1 60); do
    version_en $1 | grep -q "\"$2\"" && break
    echo -n "."; sleep 3
  done
  echo; estado; version_en $1
}
```

Paso 5 · Primera ejecución y aprobación

En AWS real, una pipeline nueva arranca sola nada más crearla. En el emulador la arrancas tú:

```
aws codepipeline start-pipeline-execution --name lab03 --query pipelineExecutionId --output text
esperar Aprobacion InProgress
```

La pipeline ha pasado Source, Build y Deploy-Pre, y se ha parado en la aprobación. Comprueba qué hay en cada entorno:

```
version_en $PRE
version_en $PRO
```

> **Punto de control 2.** Preproducción tiene la versión `1.0.0` y producción está vacía: nada llega a producción sin una aprobación.

Para aprobar necesitas el *token* de la acción de aprobación, que identifica esta ejecución concreta (así nadie aprueba por error una ejecución distinta de la que ha revisado):

```
aws codepipeline get-pipeline-state --name lab03 \
  --query "stageStates[?stageName=='Aprobacion'].actionStates[0].latestExecution.[summary,token]" --output t

aprobar() {   # uso: aprobar Approved|Rejected "comentario"
  TOKEN=$(aws codepipeline get-pipeline-state --name lab03 \
    --query "stageStates[?stageName=='Aprobacion'].actionStates[0].latestExecution.token" --output text)
  aws codepipeline put-approval-result --pipeline-name lab03 --stage-name Aprobacion \
    --action-name Aprobar --token "$TOKEN" --result "{\"status\": \"$1\", \"summary\": \"$2\"}"
}

aprobar Approved "Revisado en pre, todo correcto"
esperar_version $PRO 1.0.0
```

> **Punto de control 3.** Las cinco etapas están en `Succeeded` y producción tiene la `1.0.0`. El zip de producción es **exactamente el mismo** que se probó y se desplegó en preproducción: no se ha vuelto a construir nada.

Paso 6 · Publicar una versión nueva sin tocar la pipeline

Cambia la versión y súbela. No arrancas nada a mano: la pipeline detecta el zip nuevo por sí sola.

```
        sed -i 's/VERSION = "1.0.0"/VERSION = "1.1.0"/' app/src/app.py
        publicar
        esperar Aprobacion InProgress
        version_en $PRE
        version_en $PRO

        aws codepipeline list-pipeline-executions --pipeline-name lab03 \
          --query "pipelineExecutionSummaries[].[status,trigger.triggerType]" --output table
```

En la tabla de ejecuciones verás que la primera la arrancaste tú (`StartPipelineExecution`) y la segunda la arrancó el cambio en el origen (`PollForSourceChanges`). Preproducción ya tiene la `1.1.0` y producción sigue en la `1.0.0`.

Ahora imagina que, revisando preproducción, el equipo de negocio decide que esta versión no sale hoy. **Recházala**:

```
        aprobar Rejected "No se publica hoy: pendiente de validar con negocio"
        esperar Aprobacion Failed
        version_en $PRO
```

> **Cómo leer la tabla de estado.** Cada fila muestra la **última** vez que se ejecutó esa etapa, sea de la ejecución que sea. Por eso, después de rechazar, Deploy-Pro sigue apareciendo como `Succeeded`: es el despliegue de la `1.0.0` de la primera ejecución. Lo que manda es la versión que hay en el bucket.

> **Punto de control 4.** La etapa Aprobacion queda en `Failed`, Deploy-Pro no se ejecuta y producción sigue en la `1.0.0`. Rechazar no es un error técnico: es la pipeline funcionando como control.

Paso 7 · Una versión que rompe las pruebas

Ahora alguien sube un cambio que rompe el formato de la versión:

```
        sed -i 's/VERSION = "1.1.0"/VERSION = "1.2"/' app/src/app.py
        publicar
        esperar Build Failed
        version_en $PRE
        aws s3 cp s3://$ARTEFACTOS/build.log - | tail -15
```

> **Punto de control 5.** La etapa Build está en `Failed`, Deploy-Pre no se ha ejecutado (su `Succeeded` es de la ejecución anterior) y preproducción sigue con la `1.1.0`: el cambio roto **no ha llegado a ningún entorno**. El log del build te dice qué prueba ha fallado.

Corrígelo, publica y lleva la versión buena hasta producción:

```
        sed -i 's/VERSION = "1.2"/VERSION = "1.2.0"/' app/src/app.py
        publicar
        esperar Aprobacion InProgress
        aprobar Approved "Corrección revisada en pre"
        esperar_version $PRO 1.2.0
        version_en $PRE

        aws codepipeline list-pipeline-executions --pipeline-name lab03 \
          --query "pipelineExecutionSummaries[].[status,trigger.triggerType]" --output table
```

> **Punto de control final.** Pre y pro tienen la `1.2.0`. En el historial ves cuatro ejecuciones: dos que llegaron a producción, una detenida en la aprobación y otra detenida en el build. Has recorrido el ciclo completo de entrega continua.

Paso 8 · Cómo sería en AWS real

En una cuenta de AWS esta misma pipeline tendría tres diferencias importantes:

1. **El build recibe el artefacto de la pipeline.** El proyecto se crea con `source type=CODEPIPELINE` y `artifacts type=CODEPIPELINE`, la acción Build declara un `outputArtifacts` (por ejemplo `Paquete`) y las etapas de despliegue consumen ese paquete, no el código fuente. Así lo que se despliega es exactamente lo que se ha construido y probado.
2. **El despliegue es infraestructura como código.** Las acciones Deploy usan el proveedor `CloudFormation` con `ActionMode: CREATE_UPDATE`, una plantilla SAM dentro del paquete y un stack distinto para cada entorno (`lab03-app-pre`, `lab03-app-pro`), igual que hiciste a mano en el Lab 01.
3. **La aprobación avisa a alguien.** La acción Manual admite un tema de SNS (`NotificationArn`) y una URL de revisión (`ExternalEntityLink`), de modo que el aprobador recibe un correo con el enlace a preproducción. Y en lugar de sondear el bucket, una regla de EventBridge arranca la pipeline en cuanto llega el zip.

Así quedaría la etapa de preproducción:

```
        { "name": "Deploy-Pre", "actions": [ {
            "name": "DesplegarPre",
            "actionTypeId": { "category": "Deploy", "owner": "AWS", "provider": "CloudFormation", "version": "1" },
            "configuration": {
              "ActionMode": "CREATE_UPDATE",
              "StackName": "lab03-app-pre",
              "TemplatePath": "Paquete::empaquetada.yaml",
              "Capabilities": "CAPABILITY_IAM,CAPABILITY_AUTO_EXPAND",
              "ParameterOverrides": "{\"Entorno\":\"pre\"}",
              "RoleArn": "arn:aws:iam::123456789012:role/lab03-cfn"
            },
            "inputArtifacts": [ { "name": "Paquete" } ] } ] }
```

Reto opcional

Añade una segunda acción a la etapa Deploy-Pre, con `runOrder: 2`, que sea un build de CodeBuild con «pruebas de humo» sobre lo desplegado (por ejemplo, que descargue `app.zip` de preproducción y compruebe la versión). Así la aprobación solo se pide si preproducción, además de estar desplegada, funciona.

Preguntas de reflexión

1. ¿Por qué es importante que a producción llegue el mismo artefacto que se probó en preproducción, en lugar de reconstruirlo?
2. ¿Qué diferencia hay entre que la pipeline se detenga en Build y que se detenga en Aprobacion? ¿Quién toma la decisión en cada caso?
3. Con `executionMode: QUEUED`, ¿qué pasaría si subes dos versiones seguidas mientras una espera la aprobación? ¿Y con `SUPERSEDED`?

Limpieza

```
        cd ~/lab03
        aws codepipeline delete-pipeline --name lab03
        aws codebuild delete-project --name lab03-pruebas
        for b in $FUENTE $ARTEFACTOS $PRE $PRO; do aws s3 rm s3://$b --recursive --quiet; done
        aws s3api delete-objects --bucket $FUENTE --delete "$(aws s3api list-object-versions --bucket $FUENTE \
          --query '{Objects: Versions[].{Key:Key,VersionId:VersionId}}' --output json)" > /dev/null 2>&1
        aws cloudformation delete-stack --stack-name lab03-inicio
```

Detén el Codespace al terminar: **Code → Codespaces → ⋯ → Stop codespace**.